C++ Pattern Matching and Sorting Algorithms

📦Header Files and Constants

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

const int max_size = 100;
```

🔍Pattern Matching Algorithms

🔔Naive Pattern Matching

```cpp
void naivebais(string text, string pattern) {
    int m = pattern.length();
    int n = text.length();
    bool found = false;

    for (int shift = 0; shift <= n - m; shift++) {
        int i;
        for (i = 0; i < m; i++) {
            if (pattern[i] != text[shift + i]) {
                break;
            }
        }
        if (i == m) {
            cout << "Pattern occurs at index " << shift << endl;
            found = true;
        }
    }
    if (!found)
        cout << "Pattern not found" << endl;
}
```

🔔Rabin-Karp Pattern Matching

```
int chartoInt(char c) {
    return c - 'a';
}

int hashfunction(string str, int q) {
    int hash = 0;
    for (char c : str) {
        hash = (hash * 10 + chartoInt(c)) % q;
    }
    return hash;
}

void rabinkarp(string text, string pattern, int q) {
    int n = text.length();
    int m = pattern.length();
    bool found = false;

    int hashpattern = hashfunction(pattern, q);

    for (int i = 0; i <= n - m; i++) {
        string sub = text.substr(i, m);
        int hashtext = hashfunction(sub, q);
        if (hashtext == hashpattern && sub == pattern) {
            cout << "Pattern found at index " << i << endl;
            found = true;
        }
    }
    if (!found)
        cout << "Pattern not found" << endl;
}
```

🕐Sorting Algorithms

🔔Merge Sort (Vector)

```
void merge(vector<int>& arr, int st, int mid, int end) {
    vector<int> temp;
    int i = st;
    int j = mid + 1;

    while (i <= mid && j <= end) {
        if (arr[i] < arr[j]) {
```

```
            temp.push_back(arr[i++]);
        }
        else {
            temp.push_back(arr[j++]);
        }
    }

    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= end) temp.push_back(arr[j++]);

    for (int idx = 0; idx < temp.size(); idx++) {
        arr[st + idx] = temp[idx];
    }
}

void mergesort(vector<int>& arr, int st, int end) {
    if (st < end) {
        int mid = (st + end) / 2;
        mergesort(arr, st, mid);
        mergesort(arr, mid + 1, end);
        merge(arr, st, mid, end);
    }
}
```

🔔Bubble Sort

```
void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool isswap = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                isswap = true;
            }
        }
        if (!isswap)
            return;
    }
}
```

🔔Selection Sort

```
void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int smallestidx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[smallestidx] > arr[j]) {
                smallestidx = j;
            }
        }
        int temp = arr[smallestidx];
        arr[smallestidx] = arr[i];
        arr[i] = temp;
    }
}
```

🔔Insertion Sort

```
void insertion_sort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int curr = arr[i];
        int prev = i - 1;

        while (prev >= 0 && arr[prev] > curr) {
            arr[prev + 1] = arr[prev];
            prev--;
        }

        arr[prev + 1] = curr;
    }
}
```

⚙️Utility Functions

```
void output_array(int n, int arr[]) {
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void input_array(int& n, int arr[]) {
```

```
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```

🖥️Main Program (Menu Driven)

```
int main() {
    int n;
    int arr[max_size];
    int choice = 0;

    do {
        cout << "\n1) Naive Pattern Match\n2) Rabin-Karp\n3) Merge Sort (Vector)
\n4) Insertion Sort\n5) Selection Sort\n6) Bubble Sort\n7) Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1: {
            string str, pattern;
            cout << "Enter text: ";
            cin >> str;
            cout << "Enter pattern: ";
            cin >> pattern;
            naivebais(str, pattern);
            break;
        }

        case 2: {
            string str, pattern;
            int q;
            cout << "Enter text: ";
            cin >> str;
            cout << "Enter pattern: ";
            cin >> pattern;
            cout << "Enter prime number q: ";
            cin >> q;
            rabinkarp(str, pattern, q);
            break;
        }
```

```
        case 3: {
            input_array(n, arr);
            vector<int> vec(arr, arr + n);
            mergesort(vec, 0, n - 1);
            cout << "Sorted array: ";
            for (int val : vec) {
                cout << val << " ";
            }
            cout << endl;
            break;
        }

        case 4:
            input_array(n, arr);
            insertion_sort(arr, n);
            output_array(n, arr);
            break;

        case 5:
            input_array(n, arr);
            selection_sort(arr, n);
            output_array(n, arr);
            break;

        case 6:
            input_array(n, arr);
            bubble_sort(arr, n);
            output_array(n, arr);
            break;

        case 7:
            cout << "Exiting..." << endl;
            break;

        default:
            cout << "Invalid choice!" << endl;
        }

    } while (choice != 7);

    return 0;
}
```